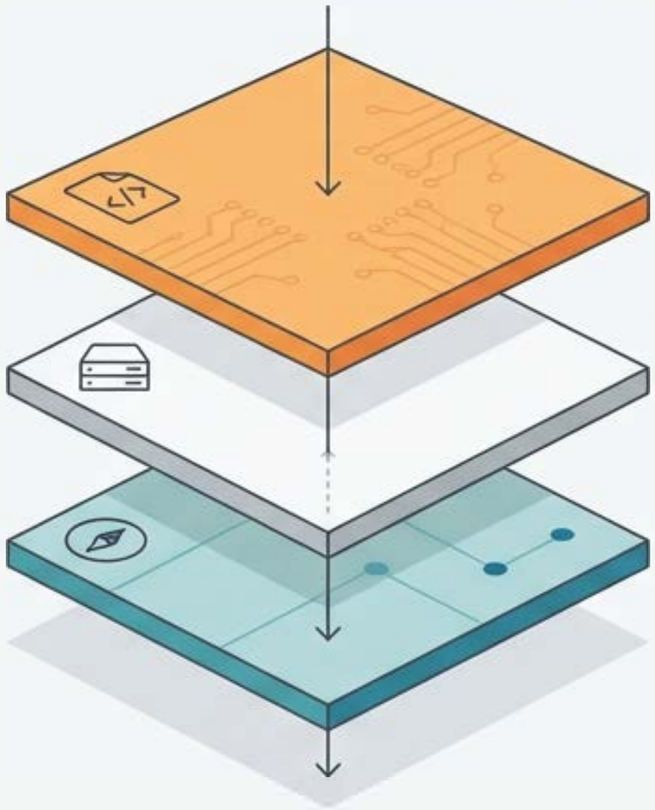
# Desarrollo Web Moderno: Arquitectura Full-Stack

## Integrando Flask, Jinja2 y FIWARE Orion

### Tech Stack



**Jinja2**
Presentation / HTML Layer

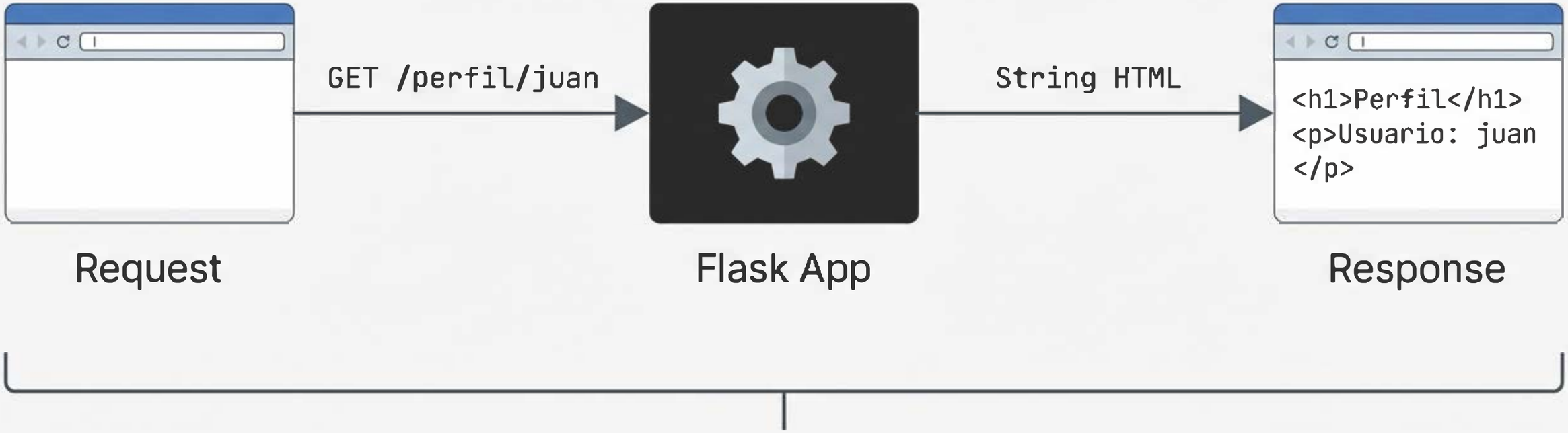
**Flask**
Logic / Routing Layer

**FIWARE Orion**
Data / Context Layer

UNIVERSIDADE DA CORUÑA

# La Web como una Llamada a Función

El desarrollo backend consiste en exponer lógica de Python a través del protocolo HTTP.



# Flask: El "Micro-Framework"

“Micro” no significa débil; significa flexible. A diferencia de Django, no impone herramientas, pero hace dos cosas excepcionalmente bien:

### Enrutamiento (Routing)

Traducir URLs en funciones de Python.

### Renderizado

Inyectar datos de Python en HTML usando Jinja2.

# Enrutamiento: El Despachador de Tráfico

```
app = Flask(__name__)

@app.route('/')
def inicio():
    return "¡Hola desde Flask!"
```

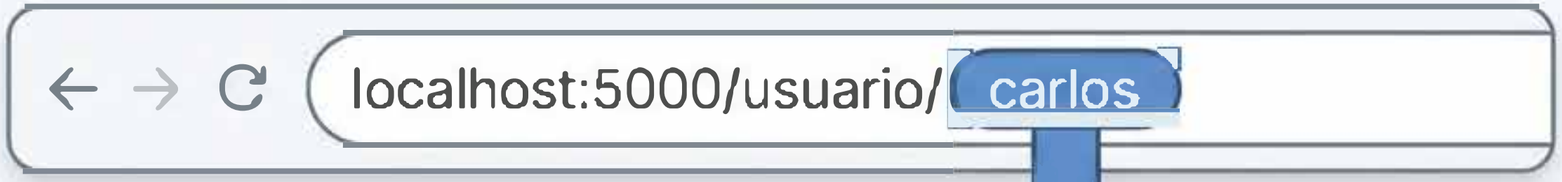
**Mapeador**: Registra la URL raíz en un diccionario interno.

**Lógica**: La función que se ejecuta cuando llega la petición.

**Respuesta**: Lo que recibe el navegador.

# URLs Dinámicas: Argumentos en la Ruta

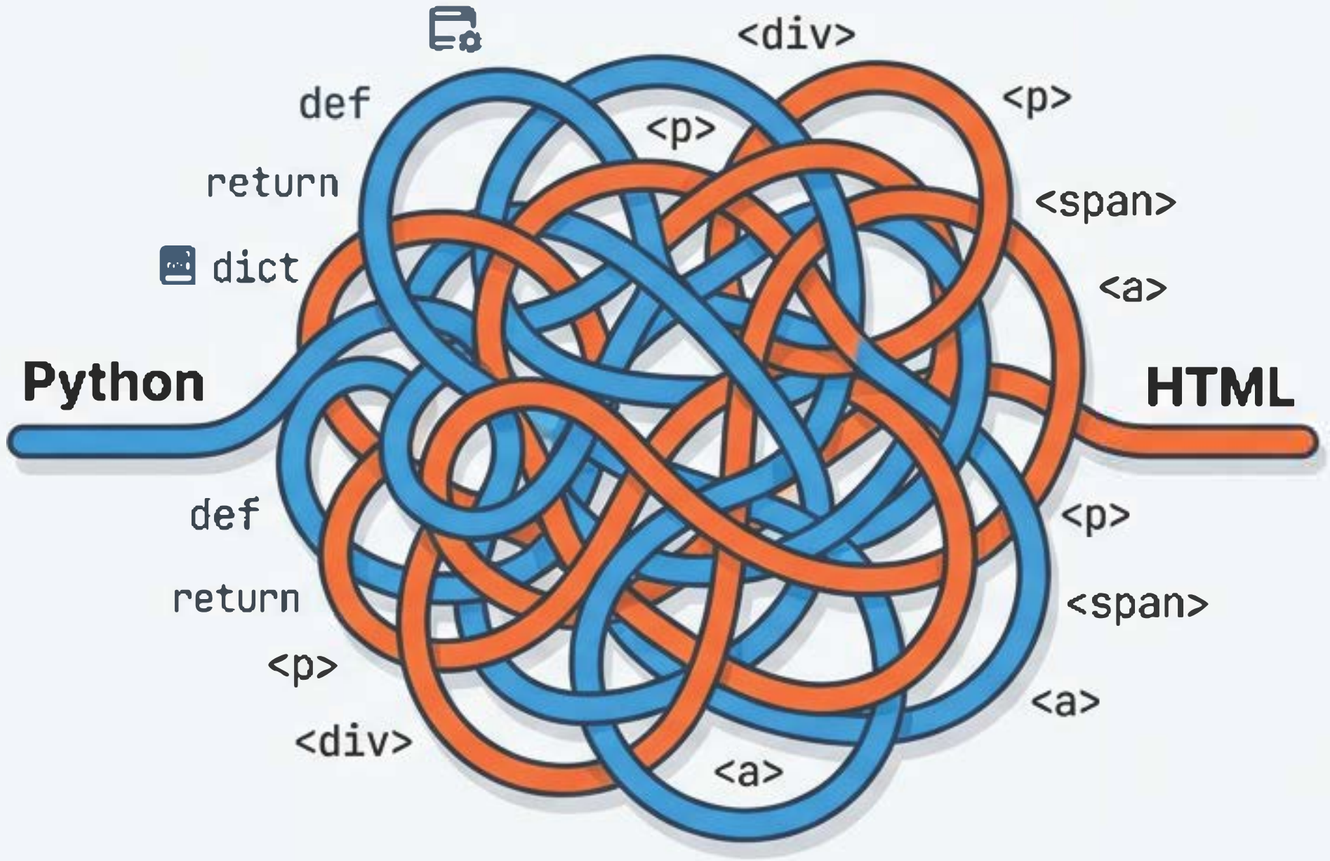
En la web, los argumentos de la función viajan **empaquetados dentro** de la URL.




```python
@app.route('/usuario/<nombre_usuario>')
def perfil(nombre_usuario):
    # nombre_usuario = "carlos"
    return f"Perfil de: {nombre_usuario}"
```

Flask extrae el valor de la URL y lo pasa automáticamente a la función.

# El Problema de la Mezcla




**Mala Práctica:** Escribir HTML dentro de los `return` de Python.

**Principio Violado:** Separation of Concerns (Separación de Responsabilidades).

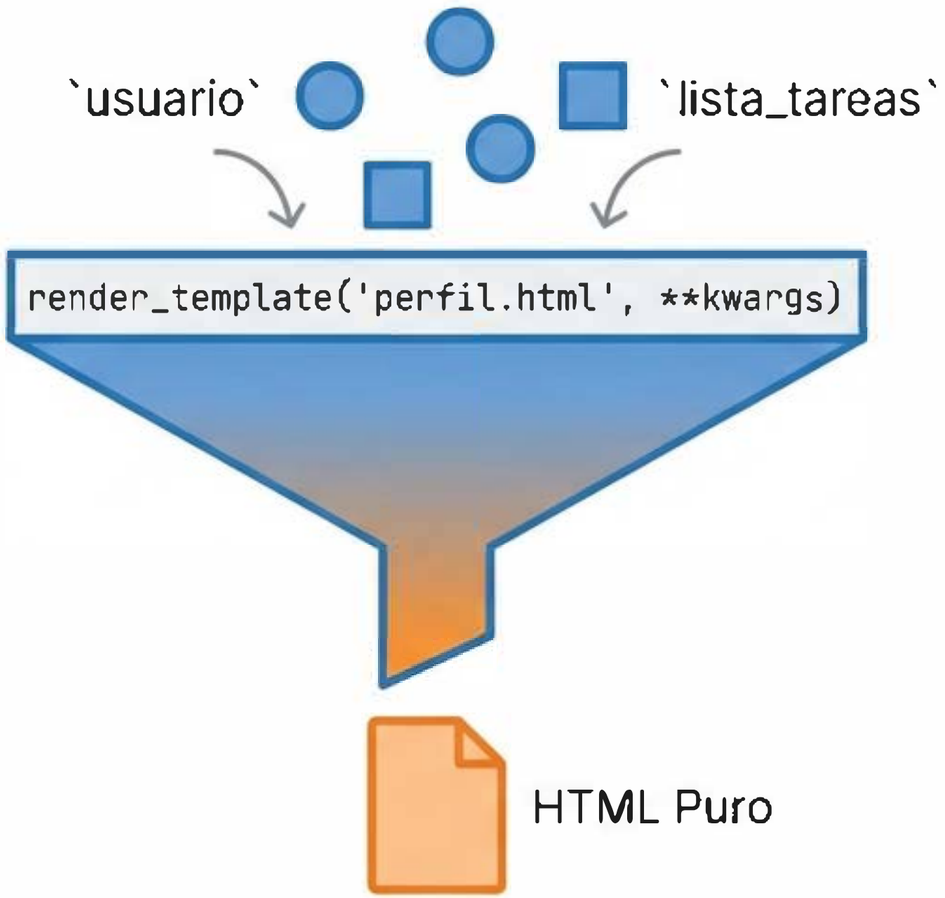
**Solución:** Motores de plantillas.

# Jinja2: HTML Hiper-Vitaminado

Un motor de plantillas diseñado para escribir HTML dinámico.

| | |
|---|---|
| {{ }} | **Expresiones.** Imprime variables de Python (como f-strings). |
| {% %} | **Declaraciones.** Lógica de control (If/For). |
| {# #} | **Comentarios.** Invisibles en el HTML final. |

# `render_template`: El Puente de Datos




```
return render_template(
    'perfil.html',
    usuario=usuario_actual,
    tareas=lista_tareas
)
```

# Lógica en la Vista

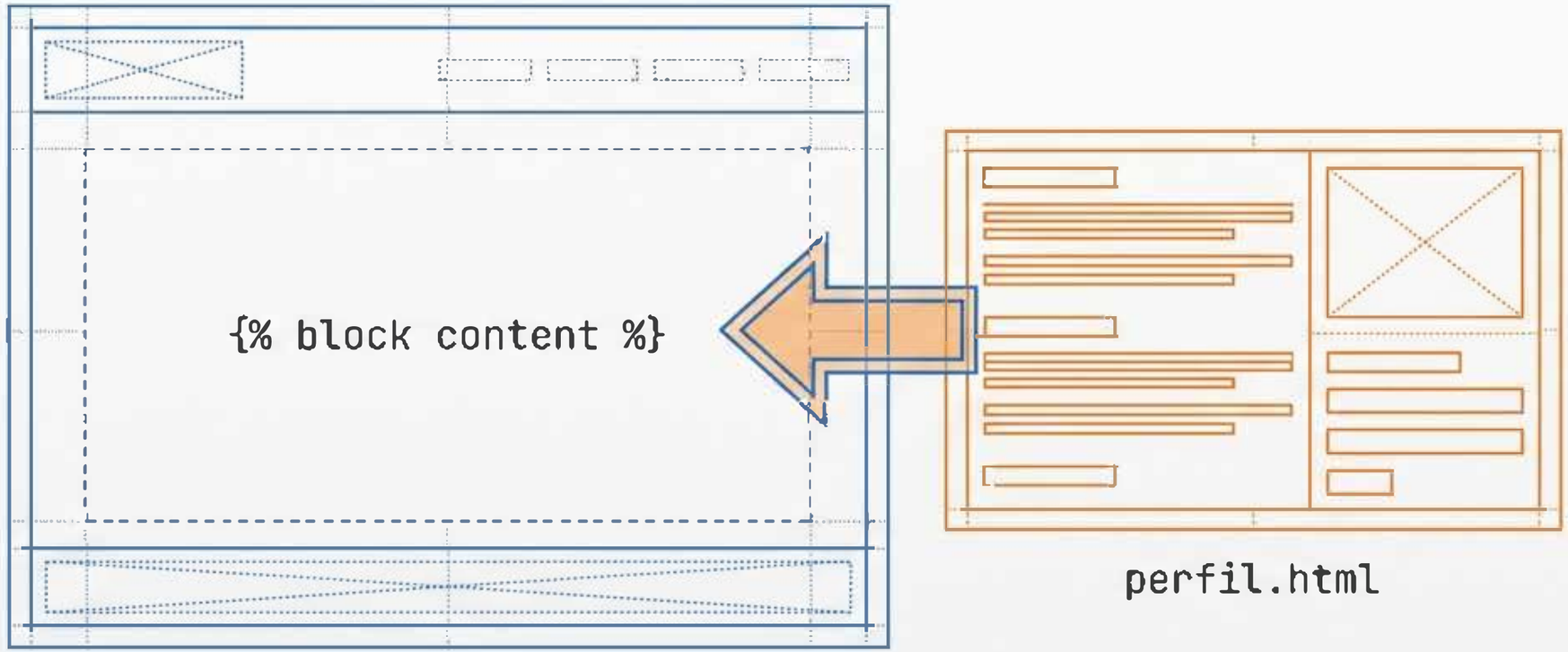
Control de flujo directamente en el HTML.

```
<h3>Tus tareas pendientes:</h3>
<ul>
    {% for tarea in tareas %}
        <li>{{ tarea }}</li>
    {% else %}
        <li>No tienes tareas pendientes.</li>
    {% endfor %}
</ul>
```

Iteración al estilo Python

# Herencia: El Principio DRY

Evitar código duplicado compartiendo estructuras comunes.



# Cambio de Paradigma: De SQL a APIs

| Desarrollo Tradicional | Arquitectura FIWARE |
|---|---|
| | |
| Bases de Datos Relacionales (SQL) | Context Broker (Orion) |
| Tablas y Filas | API REST (NGSIv2) |
| Uso de ORMs | Entidades JSON |

La aplicación Flask deja de ser un gestor de base de datos para convertirse en un **Cliente HTTP**.

# FIWARE Orion y la Librería `requests`

El estado de la aplicación “vive” en el Context Broker (Puerto 1026). Flask actúa como intermediario.

```python
import requests

ORION_URL = "http://localhost:1026/v2"
```

# Lectura de Datos (GET)

```
response = requests.get(
    f"{ORION_URL}/entities",
    params={"type": "Sensor"}
)
sensores = response.json()
# Convierte JSON NGSIv2 -> Lista de Dicts Python

return render_template('sensores.html', sensores=sensores)
```

# Visualizando Datos IoT

Source Data

```
{
  "id": "S1",
  "temperatura": {
    "value": 30,
    "type": "Number"
  }
}
```

Template Implementation

```
{% for sensor in sensores %}
  ID: {{ sensor.id }}
  Temp: {{ sensor.temperatura.value }} ºC
  {% if sensor.temperatura.value > 30 %}
    ¡Alerta!
  {% endif %}
{% endfor %}
```

# Escritura y Actualización (POST/PATCH)